LEZIONE 1 - Introduzione alla Programmazione e agli Algoritmi

Durata: 4 ore | Teoria: 2h | Esercizi: 2h

PARTE TEORICA (2 ore)

1.1 Cos'è un Programma e la Programmazione (20 min)

Definizione di Programma

Un **programma** è una sequenza di istruzioni che un computer può eseguire per risolvere un problema o svolgere un compito specifico. Possiamo pensare al programma come a una "ricetta" che il computer segue passo dopo passo.

Esempi nella vita quotidiana:

- Una ricetta di cucina è un "programma" per preparare un piatto
- Le istruzioni di montaggio di un mobile IKEA
- Le indicazioni stradali per raggiungere una destinazione

Cos'è la Programmazione?

La **programmazione** è l'attività di:

1. Analizzare un problema
2. Progettare una soluzione
3. Tradurre la soluzione in un linguaggio comprensibile al computer
4. Verificare che la soluzione funzioni correttamente

Il processo di sviluppo software:

```
Problema → Analisi → Algoritmo → Codice → Test → Soluzione
```

1.2 Hardware e Software: Panoramica Generale (20 min)

Hardware

L'**hardware** è la parte fisica del computer, ciò che possiamo toccare:

- **CPU (Central Processing Unit)**: il "cervello" che esegue le istruzioni
- **Memoria RAM**: memoria temporanea veloce
- **Hard Disk/SSD**: memoria permanente per salvare i dati
- **Periferiche di Input**: tastiera, mouse, scanner
- **Periferiche di Output**: monitor, stampante, altoparlanti

Software

Il **software** è la parte logica, l'insieme di programmi e dati:

- **Sistema Operativo**: gestisce le risorse hardware (Windows, Linux, macOS)
- **Applicazioni**: programmi per l'utente finale (browser, editor di testo)
- **Librerie**: collezioni di funzioni riutilizzabili
- **Driver**: programmi che permettono al sistema di comunicare con l'hardware

Relazione Hardware-Software

```
Utente
  ↓
Applicazioni
  ↓
Sistema Operativo
  ↓
Hardware
```

1.3 Concetto di Algoritmo (30 min)

Definizione

Un **algoritmo** è una sequenza finita e ordinata di passi elementari e non ambigui che portano alla soluzione di un problema.

Caratteristiche di un Algoritmo

1. **Finito**: deve terminare dopo un numero finito di passi
2. **Definito**: ogni passo deve essere descritto in modo preciso e non ambiguo
3. **Input**: può ricevere zero o più dati in ingresso
4. **Output**: produce uno o più risultati
5. **Efficace**: ogni operazione deve essere eseguibile in tempo finito
6. **Generale**: deve risolvere una classe di problemi, non solo un caso specifico

Esempio 1: Algoritmo per Fare il Caffè

1. Riempire la moka d'acqua fino alla valvola
2. Inserire il filtro
3. Riempire il filtro di caffè macinato
4. Avvitare la parte superiore della moka
5. Mettere la moka sul fuoco
6. Attendere che il caffè salga
7. Spegnere il fuoco
8. Versare il caffè nella tazzina

Esempio 2: Algoritmo per Calcolare la Media di Due Numeri

1. INIZIO
2. Leggi il primo numero (chiamalo A)
3. Leggi il secondo numero (chiamalo B)
4. Calcola SOMMA = A + B
5. Calcola MEDIA = SOMMA / 2

```
6. Scrivi MEDIA
7. FINE
```

Proprietà Importanti

Correttezza: l'algoritmo deve produrre il risultato corretto per tutti i possibili input validi.

Efficienza: un algoritmo efficiente utilizza meno risorse (tempo e memoria).

- **Complessità temporale**: quanto tempo impiega
- **Complessità spaziale**: quanta memoria utilizza

Determinismo: a parità di input, l'algoritmo produce sempre lo stesso output.

1.4 Esempi di Algoritmi nella Vita Quotidiana (15 min)

Esempio 3: Trovare il Massimo tra Tre Numeri

```
1. Leggi tre numeri: A, B, C
2. Poni MAX = A
3. Se B > MAX allora MAX = B
4. Se C > MAX allora MAX = C
5. Scrivi MAX
```

Esempio 4: Attraversare la Strada in Sicurezza

```
1. Avvicinati al bordo del marciapiede
2. Guarda a sinistra
3. Guarda a destra
4. Guarda di nuovo a sinistra
5. Se arrivano veicoli:
   - Aspetta che passino
   - Torna al passo 2
6. Se non ci sono veicoli:
```

```
   - Attraversa rapidamente
7. Fine
```

Esempio 5: Algoritmo di Ricerca (trovare un nome in rubrica)

```
1. Apri la rubrica telefonica
2. Stima approssimativamente la posizione del nome
3. Apri la rubrica in quella posizione
4. Se il nome cercato viene prima:
   - Sfoglia all'indietro
5. Se il nome cercato viene dopo:
   - Sfoglia in avanti
6. Se hai trovato il nome:
   - Leggi il numero di telefono
7. Altrimenti torna al passo 3
```

1.5 Linguaggi di Programmazione (20 min)

Livelli di Linguaggi

Linguaggio Macchina (Basso Livello)

- Codice binario (0 e 1)
- Direttamente eseguibile dalla CPU
- Esempio: `10110000 01100001`
- Molto difficile da scrivere e leggere

Linguaggio Assembly (Basso Livello)

- Istruzioni mnemoniche
- Più leggibile del linguaggio macchina
- Specifico per ogni architettura
- Esempio: `MOV AX, 61h`

Linguaggi di Alto Livello

- Più vicini al linguaggio umano
- Indipendenti dall'hardware
- Necessitano di compilazione o interpretazione
- Esempi: C, Python, Java, JavaScript

Classificazione dei Linguaggi

Linguaggi Compilati:

- Il codice sorgente viene tradotto tutto in una volta in linguaggio macchina
- Producono un file eseguibile
- Più veloci in esecuzione
- Esempi: C, C++, Rust

Linguaggi Interpretati:

- Il codice viene tradotto ed eseguito riga per riga
- Non producono un eseguibile
- Più lenti ma più flessibili
- Esempi: Python, JavaScript, Ruby

Processo di Compilazione (C):

```
Codice Sorgente (.c)
        ↓
  Preprocessore
        ↓
   Compilatore
        ↓
  Codice Oggetto (.o)
        ↓
     Linker
        ↓
 Eseguibile (.exe)
```

1.6 Introduzione al Linguaggio C (15 min)

Storia del C

- Sviluppato da **Dennis Ritchie** nei Bell Laboratories (1972)
- Creato per sviluppare il sistema operativo UNIX
- Influenzato dai linguaggi B e BCPL
- Base per molti linguaggi moderni (C++, C#, Java, JavaScript)

Caratteristiche del C

Vantaggi:

- **Efficiente**: produce codice veloce e ottimizzato
- **Portabile**: programmi C possono girare su diverse piattaforme
- **Potente**: accesso diretto alla memoria e all'hardware
- **Standard**: definito dagli standard ANSI/ISO
- **Diffuso**: utilizzato in sistemi operativi, embedded systems, videogiochi

Svantaggi:

- **Complesso**: richiede gestione manuale della memoria
- **Pericoloso**: possibili errori con puntatori e memoria
- **Verboso**: richiede più codice rispetto a linguaggi moderni

Dove viene utilizzato il C?

- Sistemi operativi (Linux, Windows)
- Database (MySQL, PostgreSQL)
- Linguaggi di programmazione (Python, Ruby interpreters)
- Sistemi embedded (elettrodomestici, automobili)
- Hardware drivers
- Applicazioni real-time

Primo Sguardo al C

Un semplice programma in C:

```c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Spiegazione base:

- `#include <stdio.h>`: include funzioni di input/output
- `int main()`: funzione principale del programma
- `printf()`: funzione per stampare a video
- `return 0`: indica che il programma è terminato correttamente

PARTE PRATICA (2 ore)

ESERCIZI GUIDATI

Esercizio 1: Algoritmi in Linguaggio Naturale (30 min)

Consegna: Scrivi in linguaggio naturale (italiano) gli algoritmi per risolvere i seguenti problemi. Usa la forma numerata come negli esempi visti.

1.1 Algoritmo per preparare un tè

Soluzione:
1. Riempire il bollitore d'acqua
2. Accendere il bollitore
3. Attendere che l'acqua bolla
4. Mettere la bustina di tè nella tazza
5. Versare l'acqua bollente nella tazza
6. Lasciare in infusione 3-5 minuti
7. Rimuovere la bustina

```
8. (Opzionale) Aggiungere zucchero o latte
9. Fine
```

1.2 Algoritmo per calcolare l'area di un rettangolo

```
Soluzione:
1. INIZIO
2. Leggi la BASE del rettangolo
3. Leggi l'ALTEZZA del rettangolo
4. Calcola AREA = BASE × ALTEZZA
5. Scrivi AREA
6. FINE
```

1.3 Algoritmo per determinare se un numero è pari o dispari

```
Soluzione:
1. INIZIO
2. Leggi un numero N
3. Calcola RESTO = N diviso 2 (resto della divisione)
4. Se RESTO = 0:
   - Scrivi "Il numero è pari"
5. Altrimenti:
   - Scrivi "Il numero è dispari"
6. FINE
```

Esercizio 2: Analisi di Algoritmi (30 min)

Consegna: Analizza i seguenti algoritmi e rispondi alle domande.

2.1 Algoritmo Misterioso

```
1. Leggi un numero N
2. Poni RISULTATO = 1
3. Poni CONTATORE = 1
```

```
4. Finché CONTATORE ≤ N ripeti:
   - RISULTATO = RISULTATO × CONTATORE
   - CONTATORE = CONTATORE + 1
5. Scrivi RISULTATO
```

Domande:

- Cosa fa questo algoritmo? (Calcola il fattoriale di N)
- Qual è l'input? (Un numero N)
- Qual è l'output? (N! = 1×2×3×…×N)
- L'algoritmo termina sempre? (Sì, se N ≥ 0)

2.2 Algoritmo con Problema

```
1. Leggi un numero N
2. Finché N ≠ 0 ripeti:
   - Scrivi N
   - N = N + 1
```

Domande:

- Cosa fa questo algoritmo?
- Qual è il problema? (Ciclo infinito se N ≠ 0)
- Come si può correggere? (Cambiare N = N + 1 in N = N - 1, oppure cambiare la condizione)

Esercizio 3: Creazione di Algoritmi (40 min)

Consegna: Scrivi algoritmi completi per i seguenti problemi.

3.1 Calcolare il perimetro di un rettangolo

```
Soluzione proposta:
1. INIZIO
2. Leggi BASE
3. Leggi ALTEZZA
4. Calcola PERIMETRO = 2 × (BASE + ALTEZZA)
```

5. Scrivi PERIMETRO
6. FINE

3.2 Trovare il minore tra tre numeri

Soluzione proposta:
1. INIZIO
2. Leggi tre numeri: A, B, C
3. Poni MIN = A
4. Se B < MIN allora MIN = B
5. Se C < MIN allora MIN = C
6. Scrivi MIN
7. FINE

3.3 Convertire temperatura da Celsius a Fahrenheit
Formula: F = (C × 9/5) + 32

Soluzione proposta:
1. INIZIO
2. Leggi CELSIUS
3. Calcola FAHRENHEIT = (CELSIUS × 9 / 5) + 32
4. Scrivi FAHRENHEIT
5. FINE

3.4 Calcolare il costo totale di una spesa con sconto

Soluzione proposta:
1. INIZIO
2. Leggi PREZZO_ORIGINALE
3. Leggi PERCENTUALE_SCONTO
4. Calcola SCONTO = PREZZO_ORIGINALE × PERCENTUALE_SCONTO / 100
5. Calcola PREZZO_FINALE = PREZZO_ORIGINALE - SCONTO

```
6. Scrivi PREZZO_FINALE
7. FINE
```

3.5 Verificare se un anno è bisestile
Regole: divisibile per 4 E (non divisibile per 100 O divisibile per 400)

```
Soluzione proposta:
1. INIZIO
2. Leggi ANNO
3. Se ANNO è divisibile per 400:
   - Scrivi "Anno bisestile"
4. Altrimenti se ANNO è divisibile per 100:
   - Scrivi "Anno NON bisestile"
5. Altrimenti se ANNO è divisibile per 4:
   - Scrivi "Anno bisestile"
6. Altrimenti:
   - Scrivi "Anno NON bisestile"
7. FINE
```

Esercizio 4: Installazione Ambiente di Sviluppo (20 min)

Obiettivo: Configurare l'ambiente per programmare in C

Opzione 1: Windows con MinGW

1. Scarica MinGW da: https://sourceforge.net/projects/mingw/
2. Installa selezionando "gcc" e "g++"
3. Aggiungi MinGW al PATH di sistema
4. Verifica aprendo cmd e digitando: `gcc --version`

Opzione 2: Linux

1. Apri il terminale
2. Installa gcc: `sudo apt-get install build-essential`
3. Verifica: `gcc --version`

Opzione 3: macOS

1. Installa Xcode Command Line Tools
2. Apri Terminal e digita: `xcode-select --install`
3. Verifica: `gcc --version`

IDE Consigliati:

- **Visual Studio Code**: leggero, estensioni per C/C++
- **Code::Blocks**: IDE specifico per C/C++
- **Dev-C++**: semplice per principianti (Windows)
- **CLion**: professionale (a pagamento, gratis per studenti)

Test dell'installazione:

1. Crea un file `test.c`
2. Scrivi un programma minimo (non preoccuparti della sintassi ora):

```c
#include <stdio.h>
int main() {
    printf("Funziona!\n");
    return 0;
}
```

1. Compila: `gcc test.c -o test`
2. Esegui: `./test` (Linux/Mac) o `test.exe` (Windows)

ESERCIZI DA SVOLGERE A CASA

Homework 1: Scrivi un algoritmo per ordinare tre numeri in ordine crescente.

Homework 2: Scrivi un algoritmo per calcolare il prezzo finale di un prodotto dopo l'applicazione di IVA (22%).

Homework 3: Scrivi un algoritmo che legge i voti di uno studente (5 voti) e calcola la media. Se la media è >= 6, scrive "Promosso", altrimenti "Bocciato".

Homework 4: Ricerca online esempi di programmi scritti in C e prova a capire cosa fanno (anche se non capisci ancora la sintassi).

Homework 5: Completa l'installazione dell'ambiente di sviluppo e familiarizza con l'IDE scelto.

MATERIALE DI APPROFONDIMENTO

Libri Consigliati

- "Il linguaggio C" - Kernighan & Ritchie (K&R)
- "Programmazione in C" - Kim N. King
- "C: The Complete Reference" - Herbert Schildt

Risorse Online

- https://www.learn-c.org/ - Tutorial interattivo
- https://www.cprogramming.com/ - Guide e tutorial
- https://en.cppreference.com/ - Riferimento completo del linguaggio

Video Tutorial

- Cerca su YouTube: "C programming for beginners"
- Canali consigliati: FreeCodeCamp, CS50 di Harvard

PUNTI CHIAVE DELLA LEZIONE

✓ Un **algoritmo** è una sequenza finita di passi per risolvere un problema
✓ Gli algoritmi devono essere **finiti**, **definiti** e **efficaci**
✓ La **programmazione** è il processo di tradurre algoritmi in codice
✓ Il **C** è un linguaggio potente, efficiente e ampiamente utilizzato
✓ Prima di scrivere codice, è importante **pensare all'algoritmo**
✓ La pratica è fondamentale: più algoritmi scrivi, più diventi bravo

---

PREPARAZIONE PER LA PROSSIMA LEZIONE

Nella prossima lezione studieremo:

- **Flow Chart**: rappresentazione grafica degli algoritmi
- **Costrutti fondamentali**: sequenza, selezione, iterazione
- **Pseudocodice**: linguaggio intermedio tra italiano e codice

Porta con te:

- Ambiente di sviluppo installato e funzionante
- Esercizi homework completati
- Domande e dubbi sulla lezione di oggi